OpenClaw Connecting to Robot MCP Service

OpenClaw Connecting to Robot MCP Service

- 1. Course Content
  - Course Overview
  - Learning Objectives
- 2. Preparation
- 3. MCP Protocol Introduction
- 4. OpenClaw Robot Control Communication Architecture
- 5. Demo Cases
  - 5.1 🖥️ MCP Service
  - 5.2 🛠️ MCP Tools
  - 5.3 💬 MCP Message Interface

1. Course Content

Course Overview

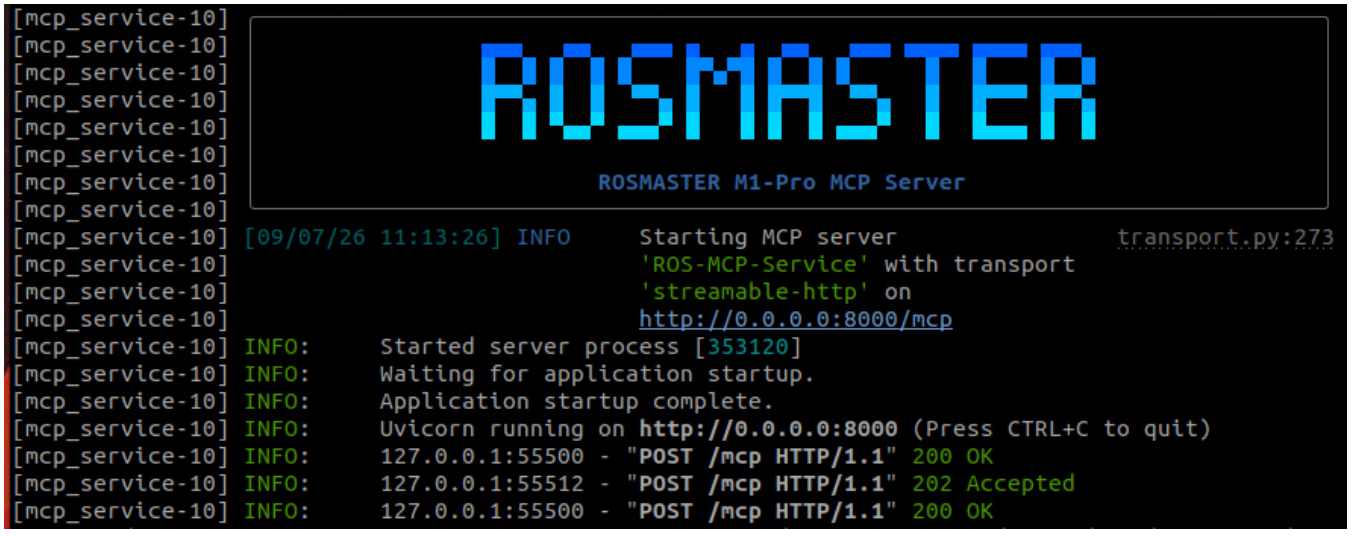
This lesson explains how to use the MCP (Model Context Protocol) interface to achieve OpenClaw's intelligent control of the robot based on the OpenClaw framework.

Learning Objectives

1. Understand the basic principles of the MCP protocol
2. Control the robot through OpenClaw conversational interaction

2. Preparation

```
ros2 launch yahboomcar_bringup mcp_service.launch.py
```

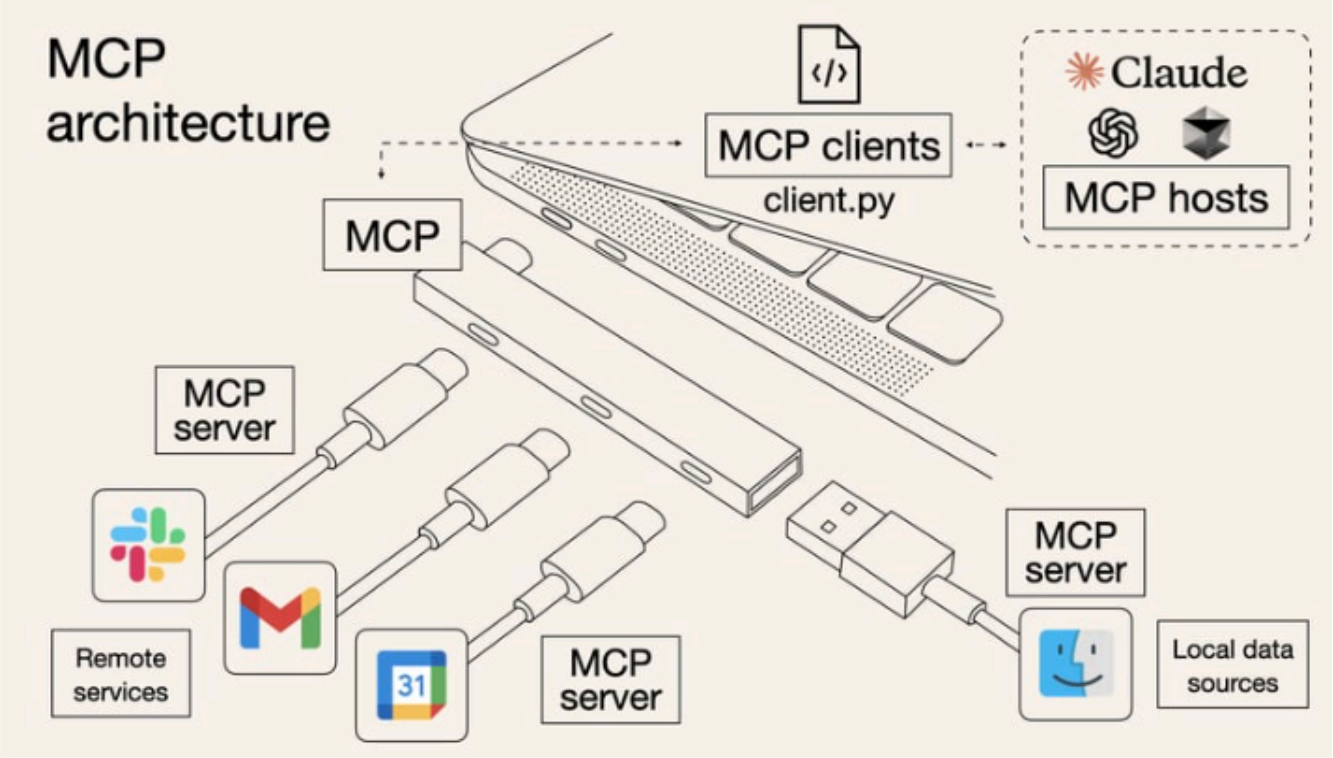


3. MCP Protocol Introduction

- MCP (Model Context Protocol) is an open protocol used to connect large language models with external

tools and services. In this system, OpenClaw converts the user's natural language instructions into specific robot operation instructions through the MCP protocol.

- After the AI model parses the user instruction, it selects the corresponding MCP Tool to execute. The MCP service converts tool calls into ROS2 Action instructions, driving hardware such as the robot chassis and robotic arm to perform actions.

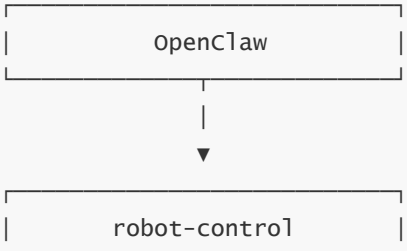


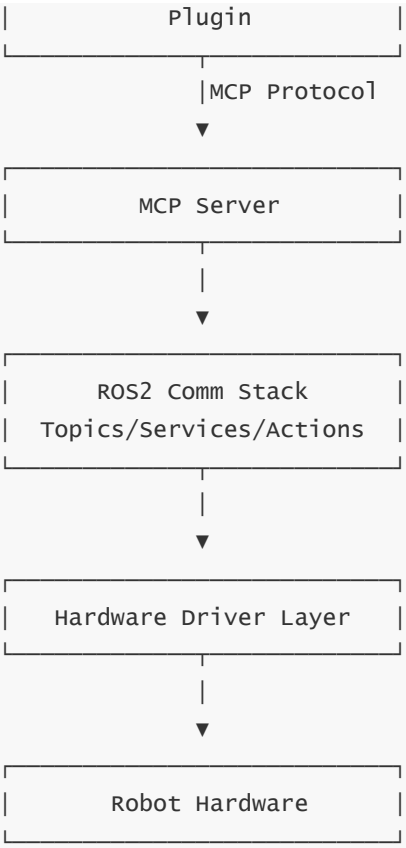
- Through the robot's MCP service interface, the robot can be connected to any MCP-supported AI model, Agent, AI tool, etc., achieving control of the robot.

4. OpenClaw Robot Control Communication Architecture

The communication architecture for OpenClaw controlling the robot is divided into four layers:

1. **User Interaction Layer**: Send natural language instructions through interaction methods such as Web Chat and WeChat
2. **AI Decision Layer**: The LLM in OpenClaw parses the user's intent and selects the corresponding MCP Tool
3. **MCP Service Layer**: Runs based on the FastMCP framework, registering tool functions such as chassis movement and visual recognition, and communicates with the robot hardware through ROS2 Action, Topic, and Service
4. **Robot Execution Layer**: The MicroROS chassis agent receives instructions and drives hardware such as the robot chassis to perform actions



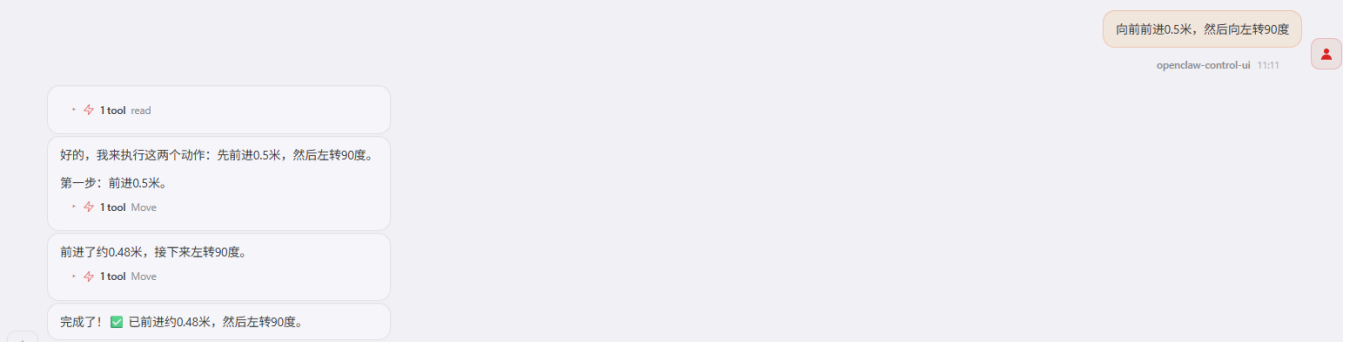


5. Demo Cases

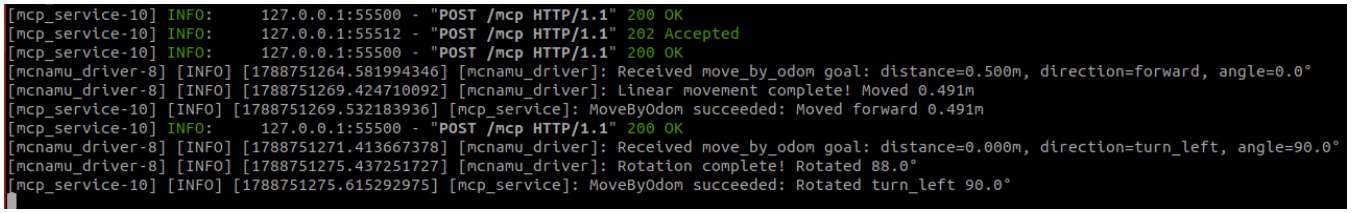
- Control the robot through conversation with OpenClaw
- On a PC on the same LAN as the robot, open the Web Chat UI and log in

> **Note**
>
> - For the interaction method with OpenClaw here, choose any one of the OpenClaw interaction methods; here we use Web Chat interaction as an example for the demonstration

- Here we use OpenClaw controlling the robotic arm as an example for the demonstration test:
  - Move forward 0.5 meters, then turn left 90 degrees



- The MCP server will indicate that it has received the corresponding MCP call request, calling the chassis control service interface through the underlying ROS2 Action



5.1 💻 MCP Service

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/mcp_service.py
```

`mcp_service.py` is the entry file of the MCP service and mainly accomplishes the following work:

1. Create a FastMCP service instance named `"ROS-MCP-Service"`
2. Create a ROS2 node through `ROSContext` and start background spin
3. Call `register_actions()` to register all robot control tools
4. Start the MCP service, listening on the specified host and port

Code Description:

- `FastMCP(name="ROS-MCP-Service")`: Create a FastMCP service instance, where `name` is the service name identifier
- `ROSContext().create_node()`: Create a ROS2 node instance, serving as a bridge for communication with the robot hardware
- `ROSContext.start_spin()`: Start the ROS2 background spin thread, continuously handling callbacks for topics, services, and actions
- `signal.signal(signal.SIGINT, signal_handler)`: Register a signal handler to capture interrupt signals such as Ctrl+C, ensuring ROS2 resources can be properly released when the service shuts down
- `register_actions(mcp, action_controller)`: Register all robot control tools (chassis movement, visual recognition, robotic arm control, etc.) into the MCP service
- `mcp_service.run(host="0.0.0.0", port=8000, transport="streamable-http")`: Start MCP service listening, where `host` is the listening address (`0.0.0.0` means allowing LAN access), `port` is the service port, and `transport` is the transport protocol

Service Parameter Description:

- `host`: Service listening address; `"0.0.0.0"` means listening on all network interfaces and allowing access from other devices on the LAN; if only local access is needed, it can be set to `"127.0.0.1"`
- `port`: Service port number, default `8000`. The robot-control plugin connects to this service via `http://localhost:8000/mcp`
- `transport`: Transport protocol; `"streamable-http"` means using the streamable HTTP protocol, supporting request-response mode and streaming transmission, suitable for real-time communication in robot control scenarios
- `ROSContext`: ROS2 context manager, encapsulating node creation, spin thread management, and

resource release, ensuring correct integration of the MCP service with the ROS2 communication stack

```
from fastmcp import FastMCP
from .action_controller import ROSContext
import signal
import sys
from .mcp_tools.mcp_action import register_actions
def signal_handler(signum, frame):
    '''Handle system signals for graceful shutdown'''
    ROSContext.shutdown()
    sys.exit(0)
signal.signal(signal.SIGINT, signal_handler)
signal.signal(signal.SIGTERM, signal_handler)
def main(args=None):
    try:
        mcp_service = FastMCP(name="ROS-MCP-Service")
        action_controller = ROSContext().create_node()
        ROSContext.start_spin()
        #-------------------------------------------MCP TOOLS-----------------------------
--------------------------
        register_actions(mcp=mcp_service,action_controller=action_controller)
        mcp_service.run(host="0.0.0.0", port=8000,transport="streamable-http")

    except KeyboardInterrupt:
        print("Service interrupted by user.")
    finally:
        ROSContext.shutdown()

if __name__ == "__main__":
    main()
```

5.2 🛠 MCP Tools

Source code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/mcp_tools/mcp_actio
n.py
```

`mcp_action.py` registers MCP Tools through the `register_actions()` function

All tools follow a unified registration pattern, using the `@mcp.tool()` decorator to define the description and timeout, constraining parameter ranges through Pydantic Field, and returning the `common_response` data class. Take the chassis movement tool **Move** as an example:

Code Description:

- `@mcp.tool(description, timeout)`: MCP tool registration decorator; `description` describes the tool function so the AI model can understand when to call it, and `timeout` sets the tool execution timeout (seconds)

- `direction`: Movement direction parameter, supporting `forward`, `backward`, `turn_left`, and `turn_right`
- `distance`: Linear movement distance, unit: meters; only used for the `forward/backward` directions, set to 0.0 during rotation
- `angle`: Rotation angle, unit: degrees; only used for the `turn_left/turn_right` directions, set to 0.0 during straight movement
- `Annotated[type, description]`: Pydantic type annotation that adds description information to each parameter, helping the AI model understand the parameter meaning and value range
- `action_controller.move_by_odom(...)`: Calls the underlying ROS2 Action interface to control chassis movement through odometry feedback
- `common_response(execution_result, reason)`: Unified return format, where `execution_result` is `"success"` or `"failed"`, and `reason` contains the detailed execution result or error information

Move Tool Parameter Description:

- `direction`: Required parameter, the movement direction; options: `forward`, `backward`, `turn_left`, and `turn_right`
- `distance`: Optional parameter, linear movement distance, unit: meters; only used for the `forward/backward` directions, default value 0.0
- `angle`: Optional parameter, rotation angle, unit: degrees; only used for the `turn_left/turn_right` directions, default value 0.0
- `timeout=30.0`: Tool execution timeout, unit: seconds. Chassis movement depends on odometry feedback; if not completed within 30 seconds, it times out and returns failure
- `res[0]`: The first element of the `move_by_odom` return result, a boolean indicating whether the movement succeeded
- `res[1]`: The second element of the return result, containing detailed execution information or error description

```
    @mcp.tool(
        description='''Move the robot chassis by a specified distance or rotate by a
specified angle using odometry feedback. '''
                    '''Supported directions: forward, backward, turn_left, turn_right. '''
                    '''For linear movement (forward/backward), specify distance in meters.
'''
                    '''For rotation (turn_left/turn_right), specify angle in degrees
(positive value).''',
        timeout=30.0)
    def Move(
        direction: Annotated[str, "Movement direction: 'forward', 'backward', 'turn_left',
or 'turn_right'"],
        distance: Annotated[float, "Linear movement distance in meters. Only used when
direction is 'forward' or 'backward'. Set to 0.0 for rotation."] = 0.0,
        angle: Annotated[float, "Rotation angle in degrees. Only used when direction is
'turn_left' or 'turn_right'. Set to 0.0 for linear movement."] = 0.0
        )-> common_response:
        res = action_controller.move_by_odom(distance=distance, angle=angle,
direction=direction)
```

```
        if res[0]:
            return common_response(execution_result="success", reason=res[1])
        return common_response(execution_result="failed", reason=res[1])
```

5.3 💬 MCP Message Interface

Source code path

```
$HOME/microros_ws/src/multi_brains_pre/multi_brains_pre/message/mcp_message.py
```

`mcp_message.py` uses Python dataclasses to define the request and response message formats of MCP Tools.

All MCP Tools uniformly return `common_response`; the AI model judges whether the operation succeeded based on the `execution_result` field:

Code Description:

- `@dataclass`: Python data class decorator that automatically generates methods such as `__init__` and `__repr__`, simplifying message format definition
- `common_response`: The unified response data class for all MCP Tools
- `execution_result`: Execution result field; the value `"success"` indicates the operation succeeded and `"failed"` indicates it failed. The AI model uses this field to judge whether a retry or error report is needed
- `reason`: Detailed reason field, containing the specific result description or error information of the operation, for the AI model to understand the execution result and generate a natural language reply

```
@dataclass
class common_response:
    execution_result: str
    reason: str
```